Lab 9: Analyzing Digital Transmissions using Wireshark

Introduction to Digital Transmissions

In modern computer networks, digital transmission is the foundation of reliable communication. Data is transmitted as digital signals over network cables or wireless channels. The efficiency of this transmission depends on multiple factors, including transmission modes, encoding schemes, bandwidth usage, data rate, signal rate, and error detection mechanisms. These elements collectively determine how efficiently and accurately data moves across a network.

Wireshark, a powerful packet analyzer, allows us to observe real network traffic, measure transmission speeds, and analyze error detection mechanisms. By capturing packets in real time, we can gain valuable insights into how digital transmission works in practice.

Transmission Modes: Parallel vs. Serial

Transmission modes define how data moves from one device to another. In parallel transmission, multiple bits are sent simultaneously over multiple channels. This method is used in short-distance communication, such as transferring data inside a computer between the processor and memory. Since each bit travels on a separate wire, parallel transmission can be fast but requires more hardware and is prone to synchronization issues over long distances.

In contrast, serial transmission sends bits one at a time over a single channel. This method is widely used in networking technologies such as Ethernet, Wi-Fi, and USB, where data must travel efficiently over long distances. Serial transmission is slower per cycle than parallel transmission but is more reliable for network communication. In this lab, we will analyze ICMP (ping) and TCP packet sequences in Wireshark to observe how serial transmission works in real networks.

Data Rate vs. Signal Rate

A fundamental measure of transmission efficiency is the data rate, also called the bit rate, which refers to the number of bits transmitted per second (bps). However, data is not always transmitted as individual bits; instead, multiple bits can be encoded within a single signal change. The signal rate, or baud rate, refers to the number of signal changes per second. If a transmission system encodes multiple bits per signal change, the baud rate will be lower than the bit rate.

Signal Rate (Baud) = Bit Rate (bps) / Bits per symbol (r)

For example, if a system transmits data at 10 Mbps while encoding two bits per signal change, the baud rate will be 5 Mbaud (**note that the transmitted data was in Megabits and thus the final result is Megabaud**). Understanding this relationship is crucial for optimizing bandwidth usage in digital networks. In this lab, we will capture network traffic, extract real-time

timestamps, and compute the data rate versus signal rate to compare theoretical and observed values.

Bandwidth and Nyquist Theorem

Bandwidth refers to the data transfer rate of a network. In digital transmission, the amount of bandwidth needed depends on the data rate and the encoding scheme used. The **Nyquist Theorem** states that in order to accurately reconstruct a signal, the sampling rate must be at least twice the highest frequency present in the signal:

Sampling Rate >= 2×Maximum Frequency

For example, human voice signals typically have a maximum frequency of 4 kHz. To digitize voice signals accurately, a minimum sampling rate of 8 kHz is required. This principle is applied in various technologies, such as telephony and audio processing, to ensure high-quality signal transmission.

In this lab, we will use Wireshark to compare bandwidth consumption for different types of network traffic, such as web browsing, video streaming, and file downloads. By analyzing real-world data, we will determine how different applications affect bandwidth usage and network performance.

Error Detection in Digital Transmission

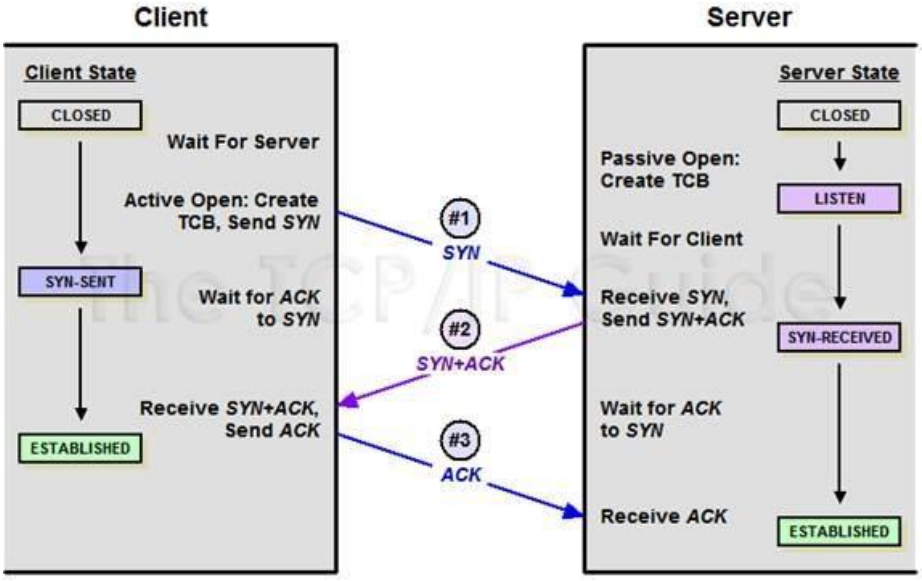
During digital transmission, errors can occur due to noise, interference, or signal degradation. To ensure data integrity, networks use error detection techniques embedded within network protocols. One of the most common error detection methods is the **Frame Check Sequence (FCS)**, found in Ethernet frames. This sequence allows the receiver to verify whether a packet has been corrupted during transmission.

Another widely used technique is the **checksum**, which is included in TCP/IP packets. The checksum algorithm calculates a unique value based on the contents of the packet. When the receiver receives the packet, it performs the same calculation and compares the result. If there is a mismatch, the packet is considered corrupted and may be discarded or retransmitted.

A more advanced error detection technique is the **Cyclic Redundancy Check (CRC)**, which is used in various network and storage systems to detect errors in data. Unlike simple checksums, CRC uses polynomial division to generate a more robust error-checking value.

In this lab, we will analyze real network packets in Wireshark to observe these error detection mechanisms in action. We will also identify packets with checksum errors and investigate whether errors occur more frequently in wired or wireless network connections.

TCP 3-way Handshake:

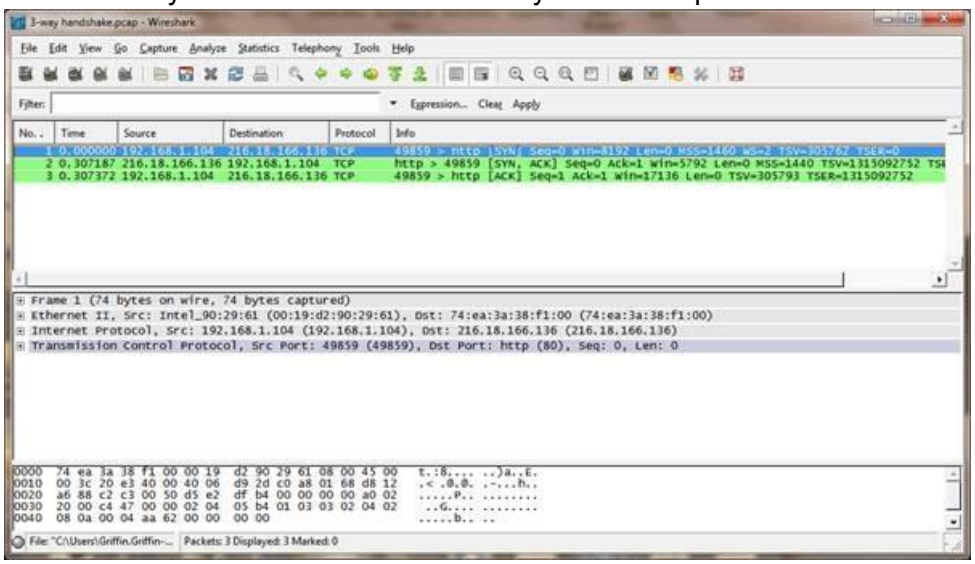


The 3-way handshake is a process which is used in TCP/IP network to establish a connection between a Client and a Server. Before the start of the actual data transmission procedure, a three-step process that involves the client and server exchanging synchronization (SYN) and acknowledgement (ACK) packets is required. Here's how it works:

1. **SYN:** The initiating computer sends a synchronize sequence number (SYN) packet to the receiving computer. The SYN packet value is set to an arbitrary number (e.g., 100) to “ask” if any open connections are available.
2. **SYN-ACK:** If the receiving computer has open ports that can accept the connection, it sends back a synchronize-acknowledge (SYN-ACK) packet to the initiating computer. The packet includes two numbers: the receiving computer’s own SYN, which can be any arbitrary number as well (e.g., 200), and the ACK number, which is the initiating computer’s SYN plus one (e.g., 101).
3. **ACK:** The initiating computer then sends an acknowledge sequence number (ACK) packet back to the receiving computer. This ACK packet is an acknowledgement of receipt of the SYN-ACK packet. The packet value is set to the receiving computer’s SYN (sent in step

two) plus one again (e.g., 201). With this final step, the connection is established, and data transmission can begin.

In Wireshark the 3-way handshake is resembled by the 3 TCP packets shown as follows:



Window Size:

In Wireshark, the TCP window size indicates how much buffer space a receiver has available to accept incoming data from the sender. It's a crucial part of TCP's flow control mechanism, preventing the sender from overwhelming the receiver. You can use the display filters to find packets with specific window size (tcp.window_size == ?).

- The window size field in the TCP header shows the number of bytes the receiver is willing to accept before needing an acknowledgment (ACK) from the sender.

TCP usually have a limitation of 2-byte window size field which has a max value of 65535. To overcome this, TCP uses window scaling. During the TCP handshake, the sender and receiver negotiate a scaling factor. The advertised window size in the packets is then multiplied by this scaling factor to get the actual calculated window size. For example, if the scaling factor is 256 and the advertised window is 2049, the calculated window size is the product, 524544.

Task 1: Data Rate vs. Signal Rate Calculation

Capture **TCP packet transmissions** to analyze data rates and calculate the signal rate (baud rate).

For this:

- Open Wireshark and start a **new packet capture**.
- Download a large file (e.g., from **Google Drive, OneDrive, or a website**) to generate sustained traffic.
- Stop the capture after a few seconds.
- Apply the *TCP* filter to display only TCP packets. Navigate to **Statistics → I/O Graph** and observe the transmission pattern.
- Navigate to **Statistics → Capture file properties** for all the details of packet transmission.

- Identify the total number of bytes transmitted and the duration of the transmission.

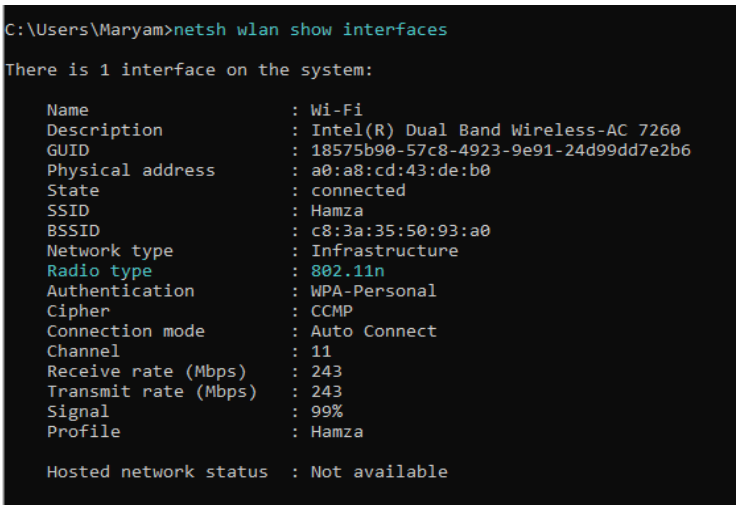
Calculate the following:

1. Data Rate (Bit Rate) = Total bits transmitted / Transmission duration
2. Packet Transmission Rate (Packets/sec) = Total packets captured / Transmission duration
3. Baud Rate (Signal Rate) using the formula given in the manual at the start.

To calculate the baud rate, you must know how many bits are transmitted per symbol, which depends on your modulation scheme (e.g., BPSK, QPSK, 16-QAM, 64-QAM, 256-QAM, etc.). You can find this information directly from your system using the following command:

netsh wlan show interfaces

Sample output of above command:



Use the table below to identify your **modulation scheme** and **bits per symbol**:

Wi-Fi Standard (Radio Type)	Typical Modulation	Bits per Symbol
802.11b	BPSK / QPSK	1–2
802.11g	16-QAM	4
802.11n	64-QAM	6
802.11ac	256-QAM	8
802.11ax (Wi-Fi 6/6E)	1024-QAM	10

Based on this information, answer the following questions:

Q1. How does the observed **data rate** compare to the **expected network speed**? (hint: you will need to check your connection speed to see the expected network speed)

Q2. If multiple bits per symbol are used, how does this affect the baud rate? (hint: you will have to find the modulation scheme for your network to find this)

Task 2: Measuring Bandwidth Usage for Different Traffic Types

Investigate how different applications consume bandwidth by analyzing **web browsing and video streaming**.

Bandwidth Usage = Amount of data transferred over time

Bandwidth (Mbps) = (Total Bytes × 8) / Time (seconds) / 1,000,000

For this:

Open Wireshark and start a new capture session.

Perform the following network activities **one at a time** while capturing packets (close all other activities on the network):

- **Browse a website** (e.g., search on Google).
- **Watch a video** (YouTube, Netflix, or Twitch).

Stop the capture and apply the appropriate filters. For web browsing use the TCP filter and for video streaming use the UDP filter (hint: you are going to need a filter like frame contains “google” or tcp contains “google”).

Open **Statistics** → **Conversations** and examine the total **Bytes and Packets exchanged** for each activity. Compare the **bandwidth usage** of web browsing and video streaming by answering the following questions.

Q1. Which type of traffic generates the **most packets** and consumes the highest bandwidth? Mathematically prove it using the given formula.

Q2. Does streaming use **larger or smaller packets** than file web browsing?

Q3. How does UDP (used in video streaming) compare to TCP (used in web browsing)?

Task 3: Identifying Error Detection Techniques in Packet Headers

Many **error detection mechanisms** such as **checksums, Frame Check Sequence (FCS), and Cyclic Redundancy Check (CRC)** exist. We will view one simple mechanism in this lab.

For this:

Open Wireshark and start capturing network traffic. Select a **TCP segment** from the captured packets and expand the **Frame Details or expand TCP**.

Locate and examine the **TCP Checksum** – which verifies data integrity in TCP packets. (hint use: tcp.checksum filter)

Apply the *tcp.analysis.flags* filter to identify packets with checksum errors. Compare error rates in **wired vs. wireless** network captures to observe any differences (you can connect an ethernet cable to your laptop/PC to test this).

Based off your observations answer the following questions:

Q1. How frequently do checksum errors occur in **wired vs. wireless networks**?

Q2. Why are **Wi-Fi packets more prone to errors** compared to Ethernet?

Submission Guidelines:

- Submit a single PDF document containing all tasks and the wireshark files to the GCR, **ensure your file name is visible in all pictures and contains your roll number**.
- Ensure that all screenshots are clear and labeled appropriately.
- Include your name, roll number, and lab number inside the **first page** of the document.
- Name the submitted document as follows: SectionLetter_RollNumber_LabNumber, for example: **J_i221234_Lab1**.
- Name the wireshark files as follows: SectionLetter_RollNumber_TaskNumber, for example: **J_i221234_T1**.
- In demo, there will be deductions based on your answers to the questions asked.

NOTE: You must show clear screenshots of every step you take. Marks will be deducted for this mistake and in some cases you may be awarded 0 (this applies to all tasks).